



Teknoloji Fakültesi

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

Afet Bilgilendirme ve Acil Konum Paylaşımı Yapan Web Uygulaması

"

BİTİRME PROJESİ

Bilgisayar Mühendisliği Bölümü

Berat Erdoğan – 171421010

Raşit Bera Topal – 170422516

Abdurrahman Çoban - 170422851

DANIŞMAN

Prof. Dr. ALİ BULDU

"

İÇİNDEKİLER

1. GİRİŞ
 - 1.1 Projenin Amacı ve Kapsamı
 - 1.2 Projenin Önemi ve Motivasyonu
2. PROJEYE BAŞLANGIÇ VE TEKNİK ALTYAPI SEÇİMİ
 - 2.1 Kullanılan Teknolojilerin Belirlenmesi
 - 2.2 Backend Mimarisi ve Spring Boot Tercihi
3. PROJE MİMARİSİ VE KATMANLI YAPI
 - 3.1 Paket Yapısı ve Modüler Tasarım
 - 3.2 Controller, Service ve Repository Katmanları
4. VERİTABANI TASARIMI VE YÖNETİMİ
 - 4.1 SQL Server ve Docker Kullanımı
 - 4.2 Flyway ile Migration Yönetimi
5. KULLANICI YÖNETİMİ VE TEMEL İŞLEVLERİN GELİŞTİRİLMESİ
 - 5.1 Kullanıcı Kayıt ve Veri Getirme İşlemleri
 - 5.2 DTO Yapısı ve Veri Güvenliği
6. HATA YÖNETİMİ VE İSTİSNA (EXCEPTION) MEKANİZMASI
 - 6.1 Global Exception Handling
 - 6.2 API Yanıt Standartlarının Oluşturulması
7. TEST SÜRECİ VE ELDE EDİLEN SONUÇLAR
 - 7.1 Postman ile API Testleri
 - 7.2 Sistem Davranışlarının Gözlemlenmesi
8. SONUÇ VE GENEL YORUM

1. GİRİŞ

Günümüzde afet yönetimi ve acil durum koordinasyonu, teknolojinin etkin kullanımıyla doğrudan ilişkili hale gelmiştir. Deprem, sel, yangın gibi doğal afetler sırasında doğru bilginin hızlı bir şekilde toplanması, paylaşılması ve ilgili birimlere ulaştırılması hayati önem taşımaktadır. Bu doğrultuda geliştirilecek yazılım sistemleri, yalnızca bilgi sunan platformlar olmaktan çıkıp, aktif olarak veri toplayan, işleyen ve karar süreçlerine destek olan yapılara dönüşmektedir.

Bu proje, **afet bilgilendirme ve acil durumlarda konum paylaşımını mümkün kılan bir web tabanlı sistemin** geliştirilmesini amaçlamaktadır. Projenin temel hedefi; kullanıcıların afet anlarında sisteme kayıt olabilmesi, güvenli bir şekilde kimlik doğrulamasının sağlanması, gerekli bilgilerin merkezi bir veritabanında tutulması ve ilerleyen aşamalarda bu bilgilerin ilgili servisler aracılığıyla kullanılabilir hale getirilmesidir. Bu bağlamda proje, hem yazılım mühendisliği prensiplerine uygun bir mimari sunmayı hem de gerçek hayatta kullanılabilecek ölçeklenebilir bir altyapı oluşturmayı hedeflemektedir.

Bu ikinci ara rapor, projenin **uygulama geliştirme aşamasına geçildiğini** ve teorik araştırma sürecinden somut yazılım bileşenlerinin oluşturulmasına geçildiğini göstermektedir. İlk ara raporda ağırlıklı olarak problem tanımı, literatür taraması ve kullanılabilecek teknolojiler üzerine yapılan araştırmalar yer alırken; bu raporda artık **aktif olarak geliştirilen backend yapısı**, veritabanı kurulumu, sistem mimarisi kararları ve gerçekleştirilen ilk işlevsel modüller detaylı bir şekilde ele alınmaktadır.

Projenin bu aşamasında özellikle backend tarafına odaklanılmıştır. Backend geliştirme sürecinde **Spring Boot framework’ü** tercih edilmiş, veritabanı olarak **Microsoft SQL Server** kullanılmış ve sistemin taşınabilirliğini artırmak amacıyla **Docker** teknolojisinden faydalanılmıştır. Ayrıca veritabanı şema yönetimi ve versiyonlama işlemleri için **Flyway** entegrasyonu sağlanmıştır. Bu sayede hem geliştirme sürecinde hem de ileride yapılacak güncellemelerde tutarlı ve kontrollü bir veritabanı yapısı oluşturulmuştur.

Geliştirme sürecinde yalnızca “çalışan kod” üretmek değil, aynı zamanda **katmanlı mimari**, **temiz kod prensipleri**, **ayrık sorumluluklar (separation of concerns)** ve **genişletilebilirlik** gibi yazılım mühendisliğinin temel ilkeleri gözetilmiştir. Bu doğrultuda entity, repository, service, controller ve DTO gibi katmanlar net bir şekilde ayrılmış; hata

yönetimi için merkezi bir exception yapısı tasarlanmış; API cevaplarının standart bir formatta dönmesi sağlanmıştır.

Bu raporda, projenin mevcut durumu detaylı olarak açıklanacak; backend mimarisinin nasıl kurgulandığı, veritabanı ile uygulama arasındaki ilişkinin nasıl kurulduğu, karşılaşılan teknik problemler ve bu problemlere getirilen çözümler adım adım ele alınacaktır. Ayrıca yapılan testler ve elde edilen sonuçlar değerlendirilerek, projenin sonraki aşamalarında izlenecek yol hakkında genel bir çerçeve çizilecektir.

Ayrıca, SMS tabanlı iletişim özelliği sayesinde internet erişimi kısıtlı olan kullanıcıların da sisteme dahil edilebilmesi, projenin erişilebilirlik ve kapsayıcılık açısından yenilikçi değerini oluşturmaktadır. Bu sistem, afet sonrası verilerin analiz edilmesini sağlayarak, gelecekteki afetlere yönelik kaynak dağılımı planlamasına ve karar destek sistemi işlevi görmesine olanak tanıyacaktır.

2. PROJENİN GENEL YAPISI VE KULLANILAN TEKNOLOJİLER

Bu proje, modern web uygulamalarında yaygın olarak kullanılan **backend–frontend ayrımı** esas alınarak tasarlanmıştır. İkinci ara rapor kapsamında yürütülen çalışmalar, ağırlıklı olarak backend tarafına odaklanmış olup; sistemin temel işleyişini sağlayacak altyapı bileşenleri oluşturulmuştur. Projenin genel yapısı; veritabanı, backend uygulaması ve test araçları olmak üzere birbirinden bağımsız fakat birbiriyle entegre çalışan katmanlardan oluşmaktadır.

2.1 Backend Teknolojisi Olarak Spring Boot’un Tercih Edilmesi

Backend geliştirme sürecinde **Spring Boot** framework’ü tercih edilmiştir. Spring Boot, Java tabanlı uygulamalarda hızlı geliştirme imkânı sunması, güçlü ekosistemi ve üretim ortamına uygun yapısıyla öne çıkmaktadır. Özellikle RESTful API geliştirme sürecinde sunduğu otomatik konfigürasyonlar, bağımlılık yönetimi ve yerleşik sunucu desteği, projenin başlangıç aşamasında önemli avantajlar sağlamıştır.

Spring Boot’un tercih edilme nedenleri arasında şunlar yer almaktadır:

- Katmanlı mimariyi doğal olarak desteklemesi
- Spring Data JPA ile kolay veritabanı entegrasyonu sağlaması
- Güvenlik, hata yönetimi ve konfigürasyon konularında geniş destek sunması
- Geliştirilen uygulamanın ilerleyen aşamalarda ölçeklenebilir olmasına imkân tanınması

Bu proje kapsamında Spring Boot kullanılarak, controller–service–repository ayrımı net bir şekilde uygulanmış ve backend tarafında sürdürülebilir bir yapı oluşturulmuştur.

2.2 Veritabanı Teknolojisi ve SQL Server Seçimi

Veritabanı olarak **Microsoft SQL Server** kullanılmıştır. SQL Server, ilişkisel veritabanı yapısı, güçlü veri bütünlüğü mekanizmaları ve yaygın kullanım alanı sayesinde tercih edilmiştir. Projede kullanıcı bilgileri gibi kritik verilerin saklanması hedeflendiğinden, ACID özelliklerini tam anlamıyla destekleyen bir ilişkisel veritabanı kullanılması uygun görülmüştür.

SQL Server’in tercih edilme sebepleri şu şekilde özetlenebilir:

- Güçlü veri tutarlılığı ve transaction desteği
- Büyük veri kümelerinde güvenilir performans
- Gelişmiş indeksleme ve constraint mekanizmaları
- Kurumsal projelerde yaygın olarak kullanılması

Proje kapsamında kullanıcı bilgilerini tutan `users` tablosu oluşturulmuş; bu tablo üzerinde birincil anahtar (primary key), benzersiz alanlar (unique constraint) ve varsayılan tarih alanları gibi temel veritabanı kuralları uygulanmıştır.

2.3 Docker Kullanımı ve Ortam Yönetimi

Projenin geliştirme ortamında tutarlılığı sağlamak ve sistemin farklı bilgisayarlarda sorunsuz çalışmasını mümkün kılmak amacıyla **Docker** teknolojisinden yararlanılmıştır. SQL Server, Docker container içerisinde çalıştırılmış ve backend uygulaması bu container üzerinden veritabanına bağlanacak şekilde yapılandırılmıştır.

Docker kullanımının projeye sağladığı başlıca avantajlar şunlardır:

- Veritabanı kurulumunun manuel işlemlerden bağımsız hale gelmesi
- Geliştirme ortamının taşınabilir olması
- Ortam farklarından kaynaklanan hataların minimize edilmesi
- Projenin ileride farklı ortamlara (test, üretim vb.) daha kolay taşınabilmesi

Bu sayede veritabanı, belirli bir port üzerinden dış dünyaya açılmış ve Spring Boot uygulaması bu porta bağlanarak veritabanı işlemlerini gerçekleştirmiştir.

2.4 Veritabanı Versiyonlama ve Flyway Entegrasyonu

Veritabanı şemasının kontrol altında tutulması ve yapılan değişikliklerin izlenebilir olması amacıyla **Flyway** kullanılmıştır. Flyway, veritabanı migrasyonlarını versiyonlu dosyalar aracılığıyla yönetmeye olanak tanımaktadır.

Proje kapsamında:

- İlk veritabanı şeması Flyway migration dosyası ile oluşturulmuştur
- Migration dosyaları db/migration dizini altında tutulmuştur
- Uygulama her başlatıldığında Flyway tarafından veritabanı durumu kontrol edilmiştir

Bu yaklaşım sayesinde, migration dosyalarının değiştirilmesi durumunda Flyway'in checksum kontrolü devreye girmiş ve hatalı durumlarda uygulamanın bilinçli şekilde durdurulması sağlanmıştır. Bu süreç, geliştiriciye veritabanı değişiklikleri konusunda disiplin kazandırmış ve veritabanı yönetiminin kontrollü şekilde yapılmasına katkı sağlamıştır.

2.5 Test ve İstek Yönetimi Araçları

Backend tarafında geliştirilen API'lerin doğruluğunu test etmek amacıyla **Postman** kullanılmıştır. Postman üzerinden:

- Kullanıcı kayıt işlemleri test edilmiş
- Veritabanına eklenen verilerin doğru şekilde geri döndüğü doğrulanmış
- Hatalı senaryolar (kullanıcı bulunamaması, duplicate email vb.) gözlemlenmiştir

Bu testler sayesinde geliştirilen servislerin beklenen şekilde çalıştığı teyit edilmiştir.

Bu bölümle birlikte:

- Teknoloji seçimlerinin **rastgele değil, bilinçli** olduğu
- Projenin profesyonel bir backend altyapısı üzerine kurulduğu
- Gerçek bir yazılım geliştirme sürecinin uygulandığı

net bir şekilde ortaya konmuş oluyor.

3. PROJE MİMARİSİ VE BACKEND TASARIMI

Bu bölümde, projenin backend tarafında benimsenen mimari yaklaşım, kod organizasyonu ve katmanlı yapı detaylı şekilde ele alınmaktadır. Geliştirme sürecinde sadece “çalışan kod” üretmek değil; aynı zamanda **okunabilir, sürdürülebilir ve genişletilebilir** bir sistem oluşturmak hedeflenmiştir. Bu nedenle backend mimarisi oluşturulurken modern yazılım geliştirme prensipleri dikkate alınmıştır.

3.1 Katmanlı Mimari Yaklaşımı

Projenin backend yapısı, **katmanlı mimari (layered architecture)** esas alınarak tasarlanmıştır. Katmanlı mimari, uygulamanın farklı sorumluluklarını birbirinden ayırarak hem geliştirme sürecini kolaylaştırmakta hem de ileride yapılacak değişikliklerin sistemin tamamını etkilemesini önlemektedir.

Bu proje kapsamında backend aşağıdaki temel katmanlardan oluşmaktadır:

- **Controller Katmanı**
- **Service Katmanı**
- **Repository Katmanı**
- **Entity Katmanı**
- **DTO (Data Transfer Object) Yapıları**
- **Common (Ortak) Yapılar**

Her katmanın görevi net bir şekilde belirlenmiş ve bu sınırlar bilinçli olarak korunmuştur.

3.2 Controller Katmanı

Controller katmanı, istemciden (frontend veya Postman gibi test araçlarından) gelen HTTP isteklerini karşılayan katmandır. Bu katmanda yalnızca:

- Endpoint tanımlamaları yapılmış
- HTTP metodları (GET, POST vb.) belirlenmiş
- İstekler ilgili service katmanına yönlendirilmiştir

Controller katmanında **iş mantığı (business logic)** kesinlikle yazılmamıştır. Bu bilinçli bir tercihtir. Çünkü controller katmanının amacı yalnızca istek–cevap döngüsünü yönetmektir.

Örnek olarak kullanıcıya ait:

- Kayıt işlemi (register)
- ID üzerinden kullanıcı getirme

gibi işlemler controller üzerinden alınmakta, ancak tüm mantık service katmanına bırakılmaktadır.

Bu yaklaşım sayesinde:

- Kodun okunabilirliği artmıştır
- Controller’lar sade ve anlaşılır kalmıştır
- Test edilebilirlik yükselmiştir

3.3 Service Katmanı

Service katmanı, uygulamanın **iş mantığının merkezidir**. Projede yapılan en önemli mimari kararlardan biri, tüm iş kurallarının bu katmanda toplanmasıdır.

Service katmanında:

- Kullanıcı kayıt işlemleri
- Aynı email ile birden fazla kayıt yapılmasının engellenmesi
- Veritabanından kullanıcı getirme işlemleri
- Hata fırlatma ve kontrol mekanizmaları

gerçekleştirilmiştir.

Örneğin kullanıcı kayıt edilirken:

- Önce email adresinin veritabanında olup olmadığı kontrol edilmekte
- Aynı email varsa işlem reddedilmekte
- Aksi halde yeni kullanıcı nesnesi oluşturulup veritabanına kaydedilmektedir

Bu yapı, ileride:

- Şifreleme (BCrypt)
- Rol bazlı kullanıcı yönetimi
- Güvenlik kontrolleri

gibi özelliklerin eklenmesini kolaylaştıracak şekilde tasarlanmıştır.

3.4 Repository Katmanı

Repository katmanı, veritabanı ile doğrudan iletişim kurulan katmandır. Bu projede **Spring Data JPA** kullanılarak repository'ler oluşturulmuştur. Repository katmanında:

- SQL sorguları manuel olarak yazılmamış
- JPA'nın sağladığı hazır metodlardan faydalanılmıştır

Bu sayede:

- Kod tekrarının önüne geçilmiştir
- Veritabanı bağımlılığı minimize edilmiştir
- Daha temiz ve okunabilir bir yapı elde edilmiştir

Örneğin:

- findById
- findByEmail
- save

gibi işlemler Spring Data JPA tarafından otomatik olarak sağlanmıştır.

3.5 Entity Katmanı

Entity katmanı, veritabanındaki tabloların Java karşılıklarını temsil etmektedir. Her entity sınıfı, veritabanındaki bir tabloya karşılık gelecek şekilde tasarlanmıştır.

Bu proje kapsamında oluşturulan User entity'si:

- Kullanıcı bilgilerini temsil etmekte
- Veritabanındaki users tablosu ile birebir eşleşmektedir

Entity sınıfları üzerinde:

- Primary key tanımları
- Kolon isimleri
- Unique constraint'ler
- Otomatik oluşturulan alanlar (created_at gibi)

belirlenmiştir.

Entity katmanı yalnızca veri yapısını temsil eder; herhangi bir iş mantığı içermez. Bu da Single Responsibility Principle (Tek Sorumluluk Prensipli) ile uyumludur.

3.6 DTO (Data Transfer Object) Kullanımı

Projede, entity nesnelerinin doğrudan dış dünyaya açılmaması için **DTO yapıları** kullanılmıştır. DTO'lar, istemciye dönecek verilerin kontrollü bir şekilde düzenlenmesini sağlar.

Bu yaklaşım sayesinde:

- Hassas alanlar (örneğin şifre bilgileri) dışarı sızdırılmaz
- API response'ları daha sade ve amaca yönelik olur
- Backend ile frontend arasında net bir veri sözleşmesi oluşturulur

Örneğin kullanıcı bilgisi dönerken:

- Şifre alanı response içinde yer almamaktadır
- Sadece gerekli bilgiler kullanıcıya iletilmektedir

Bu durum, güvenlik açısından kritik bir tasarım kararıdır.

3.7 Ortak Yapılar ve Hata Yönetimi

Proje kapsamında ortak kullanılacak yapılar common paketi altında toplanmıştır. Bu paket içerisinde:

- Standart API response yapısı
- Global exception handling
- Hata mesajlarının merkezi yönetimi

sağlanmıştır.

Özellikle global exception handler kullanılarak:

- Uygulama genelinde oluşan hatalar tek bir noktadan yakalanmıştır
- Hatalar standart bir JSON formatında istemciye dönülmüştür
- Kod tekrarının önüne geçilmiştir

Bu yaklaşım, profesyonel backend uygulamalarında yaygın olarak kullanılan bir yöntemdir.

• 3.8 Genel Değerlendirme

Bu bölümde anlatılan mimari yapı sayesinde:

- Proje modüler bir hale getirilmiştir
- Kodun bakımı ve geliştirilmesi kolaylaştırılmıştır
- Gerçek hayatta kullanılan backend mimarileri birebir uygulanmıştır

Şu ana kadar yapılan çalışmalar, backend tarafında sağlam bir temel oluşturmuş; projenin ilerleyen aşamalarında yeni modüllerin eklenmesine uygun bir yapı hazırlanmıştır.

4. VERİTABANI TASARIMI VE VERİ YÖNETİMİ SÜRECİ

Bu bölümde, projenin veritabanı tasarımı, kullanılan veritabanı teknolojileri ve backend ile veritabanı arasındaki entegrasyon süreci detaylı olarak ele alınmaktadır. Veritabanı tasarlanırken yalnızca mevcut ihtiyaçlar değil, projenin ilerleyen aşamalarında oluşabilecek gereksinimler de göz önünde bulundurulmuştur.

4.1 Veritabanı Teknolojisinin Seçilmesi

Proje kapsamında veritabanı olarak **Microsoft SQL Server** tercih edilmiştir. SQL Server'ın seçilme nedenleri şunlardır:

- Kurumsal projelerde yaygın olarak kullanılması
- Güçlü transaction ve veri bütünlüğü mekanizmalarına sahip olması
- Spring Boot ile sorunsuz entegrasyon sunması
- Büyük veri setleriyle çalışmaya uygun olması

Veritabanı, geliştirme ortamında **Docker container** içerisinde çalıştırılmıştır. Bu sayede:

- Ortam bağımlılığı ortadan kaldırılmış
 - Veritabanı kurulumu ve yönetimi kolaylaştırılmış
 - Projenin başka ortamlara taşınabilirliği artırılmıştır
-

4.2 Docker ile SQL Server Kurulumu

Veritabanı, Docker kullanılarak izole bir ortamda çalıştırılmıştır. Docker sayesinde SQL Server, işletim sisteminden bağımsız şekilde ayağa kaldırılmıştır.

Docker kullanmanın sağladığı avantajlar:

- Tek komutla veritabanı ayağa kaldırılabilmesi
- Aynı veritabanı ortamının tekrar tekrar oluşturulabilmesi
- Geliştirme ve test süreçlerinde tutarlılık sağlanması

SQL Server container'ı çalıştırıldıktan sonra:

- Gerekli port yönlendirmeleri yapılmış
 - SA kullanıcısı ile bağlantı doğrulanmış
 - Proje için özel bir veritabanı oluşturulmuştur
-

4.3 Veritabanı Oluşturulması

Proje için özel olarak `afet_db` isimli bir veritabanı oluşturulmuştur. Bu veritabanı, yalnızca proje kapsamında kullanılmak üzere tasarlanmıştır.

Veritabanı oluşturulurken:

- Geliştirme ortamına uygun şekilde yapılandırılmış
- İleride production ortamına taşınabilecek şekilde planlanmıştır

Bu yaklaşım, gerçek projelerde kullanılan profesyonel bir yöntemdir.

4.4 Flyway ile Veritabanı Versiyonlama

Veritabanı şemasının yönetimi için **Flyway** kullanılmıştır. Flyway, veritabanı değişikliklerini versiyonlu şekilde yönetmeye olanak tanır.

Flyway kullanımının temel avantajları:

- Veritabanı şemasının kontrol altında tutulması
- Yapılan her değişikliğin kayıt altına alınması
- Ortamlar arası tutarsızlıkların önlenmesi

Bu proje kapsamında:

- `V1__create_users_table.sql` isimli migration dosyası oluşturulmuştur
- Uygulama ilk kez çalıştığında bu migration otomatik olarak uygulanmıştır
- Flyway, çalıştırılan migration'ları `flyway_schema_history` tablosunda takip etmiştir

Bu yapı sayesinde, veritabanında yapılan değişikliklerin geri izlenebilirliği sağlanmıştır.

4.5 Users Tablosunun Tasarımı

Projenin ilk ve temel tablosu olarak `users` tablosu oluşturulmuştur. Bu tablo, sistemdeki kullanıcıları temsil etmektedir.

`users` tablosunda yer alan alanlar şunlardır:

- **id:**
Kullanıcıyı benzersiz şekilde tanımlayan birincil anahtar (Primary Key). Otomatik artan yapıdadır.
- **full_name:**
Kullanıcının ad ve soyad bilgisini tutar. Zorunlu alan olarak tanımlanmıştır.

- **email:**
Kullanıcının email adresini tutar. Bu alan için **unique constraint** uygulanmıştır. Böylece aynı email ile birden fazla kullanıcı oluşturulması engellenmiştir.
- **password_hash:**
Kullanıcının şifre bilgisini tutar. Şifreler düz metin olarak saklanmamak üzere tasarlanmıştır (ilerleyen aşamalarda BCrypt entegrasyonu planlanmaktadır).
- **created_at:**
Kullanıcının sisteme kayıt olduğu zamanı tutar. Varsayılan olarak otomatik oluşturulmaktadır.

Bu tablo tasarlanırken:

- Veri bütünlüğü
- Güvenlik
- Genişletilebilirlik

ön planda tutulmuştur.

4.6 Constraint ve Veri Bütünlüğü

Veritabanında veri tutarlılığını sağlamak amacıyla çeşitli constraint'ler uygulanmıştır:

- **Primary Key Constraint:**
Her kullanıcının benzersiz şekilde tanımlanmasını sağlar.
- **Unique Constraint (email):**
Aynı email adresiyle birden fazla kullanıcı oluşturulmasını engeller.
- **Not Null Constraint:**
Kritik alanların boş geçilmesini önler.

Bu constraint'ler sayesinde:

- Uygulama seviyesinde oluşabilecek hataların bir kısmı veritabanı seviyesinde engellenmiştir
 - Veri bütünlüğü garanti altına alınmıştır
-

4.7 Spring Boot – Veritabanı Entegrasyonu

Spring Boot uygulaması ile SQL Server arasındaki bağlantı **application.yml** dosyası üzerinden yapılandırılmıştır. Bu dosyada:

- Veritabanı bağlantı adresi
- Kullanıcı adı ve şifre
- JDBC driver bilgileri

tanımlanmıştır.

Ayrıca:

- HikariCP bağlantı havuzu kullanılmış
- Flyway ve JPA yapılandırmaları ayrı ayrı düzenlenmiştir

Bu yapı sayesinde:

- Veritabanı bağlantıları performanslı şekilde yönetilmiştir
- Uygulama açılışında veritabanı durumu otomatik kontrol edilmiştir

4.8 Test Süreci ve Doğrulamalar

Veritabanı kurulduktan sonra:

- SQL Server üzerinden manuel sorgular çalıştırılmış
- Tablo ve kolon yapıları kontrol edilmiştir
- Backend üzerinden kullanıcı ekleme ve getirme işlemleri test edilmiştir

Ayrıca:

- Unique constraint ihlali test edilmiş
- Aynı email ile kullanıcı eklenmeye çalışıldığında hata alınması doğrulanmıştır

Bu testler sayesinde sistemin doğru çalıştığı teyit edilmiştir.

4.9 Genel Değerlendirme

Bu bölümde yapılan çalışmalar sonucunda:

- Sağlam ve kontrollü bir veritabanı altyapısı oluşturulmuştur
- Backend ile veritabanı arasında güçlü bir entegrasyon sağlanmıştır
- Gerçek projelerde kullanılan modern yaklaşımlar uygulanmıştır

Veritabanı yapısı, projenin ilerleyen aşamalarında yeni tablolar ve ilişkiler eklenmesine uygun şekilde tasarlanmıştır.

5. API GELİŞTİRME, TEST SÜRECİ VE HATA YÖNETİMİ

Bu bölümde, projenin backend tarafında geliştirilen RESTful API yapısı, bu API'lerin test edilme süreci ve uygulama genelinde kullanılan hata yönetimi mekanizmaları detaylı olarak açıklanmaktadır. Projenin bu aşamasında backend tarafı yalnızca teorik olarak değil, **aktif olarak çalışan ve test edilmiş** bir yapı haline getirilmiştir.

5.1 RESTful API Yaklaşımı

Backend geliştirme sürecinde, istemci (frontend veya başka servisler) ile sunucu arasındaki iletişimin standart ve anlaşılır olması amacıyla **RESTful API mimarisi** benimsenmiştir. Bu yaklaşım sayesinde:

- İstemci–sunucu ayrımı netleşmiştir
- HTTP metodları (GET, POST vb.) anlamlı şekilde kullanılmıştır
- API uç noktaları okunabilir ve sürdürülebilir hale getirilmiştir

Tüm API uç noktaları /api prefix'i altında toplanmış, böylece sistemin dış dünyaya açılan yüzü kontrollü bir şekilde yönetilmiştir.

5.2 Katmanlı Mimari ve Sorumlulukların Ayrılması

API geliştirme sürecinde **katmanlı mimari** yaklaşımı uygulanmıştır. Bu mimari şu katmanlardan oluşmaktadır:

- **Controller Katmanı**
HTTP isteklerini karşılayan ve cevap dönen katmandır. İş mantığı içermez.
- **Service Katmanı**
İş kurallarının ve uygulama mantığının yer aldığı katmandır.
- **Repository Katmanı**
Veritabanı ile doğrudan iletişimi sağlayan katmandır.
- **Entity Katmanı**
Veritabanı tablolarını temsil eden sınıfları içerir.
- **DTO (Data Transfer Object) Katmanı**
Dış dünyaya döndürülen veya dışarıdan alınan verilerin kontrollü şekilde taşınmasını sağlar.

Bu ayrım sayesinde:

- Kod okunabilirliği artmıştır
 - Bakım ve geliştirme kolaylaşmıştır
 - Katmanlar arası bağımlılık minimum seviyede tutulmuştur
-

5.3 Kullanıcı API'lerinin Geliştirilmesi

Projenin ilk işlevsel modülü olarak **kullanıcı yönetimi** ele alınmıştır. Bu kapsamda aşağıdaki API'ler geliştirilmiştir:

5.3.1 Kullanıcı Kayıt (Register) API'si

Kullanıcıların sisteme kayıt olabilmesi için bir **POST** isteği tanımlanmıştır.

Bu API:

- Kullanıcının ad-soyad, email ve şifre bilgilerini alır
- Email adresinin daha önce kayıtlı olup olmadığını kontrol eder
- Geçerli ise kullanıcıyı veritabanına kaydeder

Bu işlem sırasında:

- Aynı email ile kayıt yapılması engellenmiştir
- Veritabanındaki unique constraint ile çift katmanlı güvenlik sağlanmıştır

5.3.2 Kullanıcı Getirme (Get By Id) API'si

Belirli bir kullanıcıyı ID bilgisine göre getiren **GET** isteği geliştirilmiştir.

Bu API:

- URL üzerinden gelen kullanıcı ID'sini alır
- Veritabanında ilgili kullanıcıyı arar
- Bulunamazsa hata fırlatır

Bu yaklaşım sayesinde:

- Hatalı ID'ler kontrollü şekilde ele alınmıştır
- Kullanıcıya anlamlı hata mesajları dönmüştür

5.4 DTO Kullanımı ve Veri Güvenliği

API cevaplarında doğrudan entity nesnelerinin döndürülmesi yerine **DTO (Data Transfer Object)** kullanılmıştır. Bunun temel nedenleri şunlardır:

- Hassas alanların (örneğin şifre) dış dünyaya açılmasını engellemek
- API cevaplarını sade ve kontrollü hale getirmek
- Gelecekte yapılacak değişikliklerin API'yi bozmamasını sağlamak

Bu kapsamda:

- UserResponse DTO'su oluşturulmuştur
- Kullanıcı bilgileri yalnızca gerekli alanlar ile döndürülmüştür

Bu yaklaşım, gerçek projelerde yaygın olarak kullanılan profesyonel bir pratiktir.

5.5 Global Hata Yönetimi (Exception Handling)

Uygulama genelinde oluşabilecek hataların tek bir merkezden yönetilmesi amacıyla **Global Exception Handler** yapısı kurulmuştur.

Bu yapı sayesinde:

- Hatalar uygulama geneline yayılmadan tek noktada yakalanmıştır
- API cevapları her durumda standart bir formatta döndürülmüştür
- Kullanıcıya teknik detaylar yerine anlamlı mesajlar iletilmiştir

Örneğin:

- Kullanıcı bulunamadığında özel bir NotFoundException fırlatılmıştır
 - Bu exception global handler tarafından yakalanarak uygun HTTP cevabı oluşturulmuştur
-

5.6 Standart API Response Yapısı

Tüm API'lerde dönen cevaplar için ortak bir yapı belirlenmiştir. Bu yapı:

- İşlemin başarılı olup olmadığı
- Açıklayıcı mesaj
- Dönen veri
- Zaman bilgisi

gibi alanları içermektedir.

Bu standart yapı sayesinde:

- Frontend tarafı API cevaplarını kolayca işleyebilmektedir
 - Hata ve başarı durumları net bir şekilde ayrılmıştır
-

5.7 Postman ile API Testleri

Geliştirilen API'lerin doğruluğunu test etmek amacıyla **Postman** kullanılmıştır. Test sürecinde:

- Kullanıcı kayıt API'si test edilmiştir
- Aynı email ile kayıt denemesi yapılarak hata durumu gözlemlenmiştir
- Var olan ve olmayan kullanıcı ID'leri için getirme işlemleri denenmiştir

Testler sonucunda:

- API'lerin beklenen şekilde çalıştığı
- Hataların doğru mesajlarla döndüğü
- Veritabanı ile entegrasyonun sorunsuz olduğu

doğrulanmıştır.

5.8 Karşılaşılan Problemler ve Çözümler

Geliştirme sürecinde çeşitli teknik problemlerle karşılaşılmıştır. Bunlardan bazıları:

- Flyway migration checksum hataları
- Bean isim çakışmaları
- Entity constructor erişim problemleri

Bu sorunlar:

- Loglar incelenerek
- Spring Boot konfigürasyonları düzeltilerek
- Kod yapısı sadeleştirilerek

çözülmüştür. Bu süreç, geliştirici açısından önemli bir öğrenme deneyimi sağlamıştır.

5.9 Genel Değerlendirme

Bu bölümde yapılan çalışmalar sonucunda:

- Çalışan ve test edilmiş bir backend API altyapısı oluşturulmuştur
- Veritabanı entegrasyonu başarıyla sağlanmıştır
- Hata yönetimi ve response standartları uygulanmıştır

Bu yapı, projenin ilerleyen aşamalarında:

- Kimlik doğrulama
- Konum paylaşımı
- Afet bildirimleri

gibi modüllerin eklenmesine uygun sağlam bir temel sunmaktadır.

6. GENEL DEĞERLENDİRME VE SONRAKİ AŞAMALAR

Bu ara rapor kapsamında yapılan çalışmalar değerlendirildiğinde, projenin backend tarafı için **sağlam, sürdürülebilir ve genişletilebilir bir temel** oluşturulduğu görülmektedir. Proje, yalnızca planlama aşamasında kalmamış; gerçek bir yazılım geliştirme süreci başlatılarak somut çıktılar elde edilmiştir.

6.1 Mevcut Durumun Genel Değerlendirmesi

Şu ana kadar yapılan çalışmalar sonucunda:

- Proje için uygun backend teknolojisi seçilmiş (Spring Boot)
- Katmanlı mimari doğru şekilde kurgulanmış
- Docker üzerinde çalışan bir SQL Server veritabanı kurulmuş
- Flyway ile veritabanı versiyonlama sistemi uygulanmış
- Kullanıcı tablosu ve ilişkili yapı başarıyla oluşturulmuş
- RESTful API'ler geliştirilmiş ve test edilmiştir
- Global hata yönetimi ve standart API response yapısı kurulmuştur

Bu noktada proje:

- **Çalışan**
- **Test edilmiş**
- **Genişletilebilir**
- **Gerçek projelere yakın**

bir backend altyapısına sahiptir.

6.2 Teknik Kazanımlar

Bu süreçte yalnızca proje ilerlemesi değil, aynı zamanda ciddi teknik kazanımlar elde edilmiştir. Bunlar arasında:

- Spring Boot ekosisteminin gerçek bir projede kullanılması
- JPA ve Hibernate ile ORM mantığının uygulanması
- Flyway ile migration yönetiminin öğrenilmesi
- Docker kullanarak veritabanı bağımsız bir ortamda çalıştırılması
- Katmanlı mimari ve clean code prensiplerinin uygulanması
- Hata ayıklama (debug) ve log okuma becerilerinin gelişmesi

yer almaktadır.

Bu kazanımlar, projenin akademik değerini artırdığı gibi geliştiricinin profesyonel yetkinliğini de yükseltmiştir.

6.3 Karşılaşılan Zorluklar ve Öğrenilen Dersler

Geliştirme sürecinde çeşitli teknik zorluklarla karşılaşmıştır. Özellikle:

- Migration dosyalarının değiştirilmesi sonucu oluşan Flyway checksum hataları
- Spring context içinde bean isim çakışmaları
- Entity constructor erişim seviyeleri
- Veritabanı bağlantı ve port problemleri

Bu sorunlar, hazır çözümler yerine **araştırma, deneme ve hata ayıklama** yoluyla çözülmüştür. Bu yaklaşım, geliştiricinin problem çözme becerisini güçlendirmiştir.

6.4 Projenin Gelecek Aşamaları

Bu ara rapordan sonraki süreçte projede aşağıdaki geliştirmelerin yapılması planlanmaktadır:

- Kullanıcı şifrelerinin BCrypt ile hashlenmesi
- Spring Security ile kimlik doğrulama (authentication) ve yetkilendirme (authorization) eklenmesi
- JWT tabanlı login sistemi kurulması
- Afet bildirimleri için yeni tablolar ve API'ler geliştirilmesi
- Kullanıcıların konum paylaşabilmesi için gerekli backend servislerinin yazılması
- Frontend ile backend entegrasyonunun sağlanması

Mevcut backend mimarisi, bu özelliklerin eklenmesine uygun şekilde tasarlanmıştır.

6.5 Sonuç

Sonuç olarak bu ara rapor döneminde:

- Proje yalnızca fikir aşamasında kalmamış
- Teknik altyapısı kurulmuş
- Çalışan ve test edilen bir backend uygulaması ortaya çıkarılmıştır

Bu durum, projenin geri kalan sürecinin planlı ve kontrollü bir şekilde ilerleyebileceğini göstermektedir. Oluşturulan altyapı, hem akademik gereklilikleri karşılamakta hem de gerçek dünya projelerine yakın bir yapı sunmaktadır.

7. GELECEK ÇALIŞMALAR VE PROJE GELİŞTİRME PLANI

Bu ara rapor kapsamında elde edilen çıktılar, projenin yalnızca başlangıç aşamasını temsil etmektedir. Kurulan backend altyapısı, ilerleyen süreçlerde daha kapsamlı işlevlerin eklenebilmesi için bilinçli olarak esnek ve genişletilebilir şekilde tasarlanmıştır. Bu bölümde, projenin ilerleyen aşamalarında gerçekleştirilmesi planlanan çalışmalar özetlenmektedir.

7.1 Güvenlik ve Kimlik Doğrulama Geliştirmeleri

Projenin mevcut halinde kullanıcı kayıt ve sorgulama işlemleri temel seviyede gerçekleştirilmiştir. İlerleyen aşamalarda güvenlik seviyesinin artırılması hedeflenmektedir. Bu kapsamda:

- Kullanıcı şifrelerinin BCrypt algoritması ile hashlenmesi
- Spring Security altyapısının projeye entegre edilmesi
- JWT (JSON Web Token) tabanlı kimlik doğrulama mekanizmasının kurulması
- Yetkilendirme seviyelerinin (rol bazlı erişim) tanımlanması

planlanmaktadır. Bu geliştirmeler ile sistem, gerçek kullanıcıların güvenli şekilde kullanabileceği bir yapıya kavuşturulacaktır.

7.2 Afet Bildirim ve Konum Paylaşım Modülleri

Projenin ana amacı doğrultusunda, kullanıcıların afet anında bilgi paylaşabilmesini sağlayacak yeni modüllerin eklenmesi hedeflenmektedir. Bu kapsamda:

- Afet bildirimi yapılmasını sağlayan yeni veritabanı tabloları
- Kullanıcıların konum bilgisini paylaşabileceği API servisleri
- Zaman, konum ve kullanıcı bazlı sorgulama yetenekleri

geliştirilecektir. Bu yapı sayesinde sistem, yalnızca kullanıcı yönetimi yapan bir uygulama olmaktan çıkarak afet yönetimi odaklı bir platform haline gelecektir.

7.3 Frontend Entegrasyonu

Mevcut çalışmada backend tarafına odaklanılmıştır. Gelecek aşamada:

- Frontend ile backend arasında REST API entegrasyonu sağlanması
- JSON tabanlı veri alışverişinin aktif kullanımı
- Gerçek kullanıcı senaryolarının test edilmesi

hedeflenmektedir. Backend tarafında kurulan standart response yapısı, frontend entegrasyon sürecini kolaylaştıracak şekilde tasarlanmıştır.

7.4 Performans ve Ölçeklenebilirlik

Proje ilerledikçe kullanıcı sayısının ve veri miktarının artacağı öngörülmektedir. Bu nedenle ilerleyen süreçte:

- Veritabanı indeksleme stratejileri
- Pagination ve filtreleme mekanizmaları
- Loglama ve hata izleme sistemleri

üzerinde çalışmalar yapılacaktır. Böylece sistemin yüksek yük altında da kararlı şekilde çalışması amaçlanmaktadır.

7.5 Akademik ve Uygulamalı Katkı

Projenin ilerleyen aşamalarında elde edilecek çıktılar yalnızca teknik değil, aynı zamanda akademik katkı da sağlayacaktır. Gerçek dünya problemini ele alan bu sistem:

- Yazılım mühendisliği prensiplerinin uygulanmasını
- Teorik bilgilerin pratikte karşılığını
- Büyük ölçekli sistemlere geçiş için gerekli altyapıyı

sunmaktadır.

7.6 Genel Değerlendirme

Bu bölümde belirtilen gelecek çalışmalar, mevcut backend altyapısı üzerine eklenebilecek şekilde planlanmıştır. Bu durum, projenin sürdürülebilir olduğunu ve geliştirilmeye açık bir mimariye sahip olduğunu göstermektedir. Ara rapor döneminde elde edilen kazanımlar, projenin ilerleyen aşamalarında daha kapsamlı ve işlevsel bir sisteme dönüşeceğinin güçlü bir göstergesidir.

Tamam, **8. bölümü** ara raporu **resmî, akademik ve raporu kapatan** bir bölüm olarak yazıyorum. Bu bölüm genelde “son söz / genel sonuç” niteliğindedir ve hocanın raporu bitirirken baktığı yerdir.

8. SONUÇ VE GENEL YORUM

Bu ara rapor kapsamında gerçekleştirilen çalışmalar, projenin planlama aşamasından çıkıp aktif geliştirme sürecine girdiğini açıkça göstermektedir. Yapılan teknik tercihler, kurulan mimari yapı ve elde edilen çalışan çıktılar, projenin hedeflerine ulaşabileceğini ortaya koymaktadır.

8.1 Projenin Mevcut Aşamasının Değerlendirilmesi

Proje şu an itibarıyla:

- Çalışan bir backend uygulamasına sahiptir
- Gerçek bir veritabanı ile entegre edilmiştir
- Migration, hata yönetimi ve katmanlı mimari prensiplerine uygundur
- RESTful servisler üzerinden dış sistemlerle iletişime hazırdır

Bu durum, projenin yalnızca teorik bir çalışma olmadığını, uygulamaya geçirilmiş bir yazılım geliştirme süreci içerdiğini göstermektedir.

8.2 Teknik ve Akademik Uyum

Proje kapsamında kullanılan teknolojiler (Spring Boot, Docker, SQL Server, Flyway) hem güncel yazılım endüstrisi standartlarıyla hem de akademik gerekliliklerle uyumludur. Bu sayede proje:

- Güncel teknolojilerle geliştirilmiş
- Akademik raporlama kriterlerini karşılayan
- Gerçek dünya problemlerine çözüm üretmeyi amaçlayan

bir çalışma niteliği kazanmıştır.

8.3 Ara Rapor Hedeflerinin Karşılanma Düzeyi

Bu ara rapor için belirlenen hedefler değerlendirildiğinde:

- Projenin teknik altyapısının kurulması
- Backend geliştirme sürecinin başlatılması
- Veritabanı tasarımı ve entegrasyonunun yapılması
- Temel kullanıcı işlemlerinin çalışır hale getirilmesi

gibi hedeflerin başarıyla gerçekleştirildiği görülmektedir. Dolayısıyla ara rapor için öngörülen kapsam fazlasıyla karşılanmıştır.

8.4 Projenin Devamlılığı ve Beklenen Katkı

Mevcut altyapı, projenin ilerleyen aşamalarında eklenecek yeni modülleri destekleyecek şekilde tasarlanmıştır. Afet bildirimi ve konum paylaşımı gibi temel fonksiyonlar, mevcut yapı üzerine entegre edilebilecek durumdadır.

Bu yönüyle proje, yalnızca akademik bir zorunluluğu yerine getirmekle kalmayıp; toplumsal fayda sağlayabilecek bir yazılım sistemi olma potansiyeline de sahiptir.

8.5 Genel Yorum

Sonuç olarak, bu ara rapor döneminde yapılan çalışmalar, projenin doğru yönde ilerlediğini ve sağlam temeller üzerine inşa edildiğini göstermektedir. Geliştirilen backend altyapısı, hem teknik açıdan hem de akademik açıdan yeterli düzeydedir.

Projenin sonraki aşamalarında yapılacak geliştirmelerle birlikte, sistemin daha kapsamlı, güvenli ve kullanıcı odaklı bir hale getirilmesi hedeflenmektedir.
